

Motzkin Neighborhood Evaluation in Iterated Local Search and Simulated Annealing for the Permutational Flow Shop Problem

Remigiusz Wojewódzki Wojciech Bożejko

`remigiusz.wojewodzki@gmail.com` `wojciech.bozejko@pwr.edu.pl`



Wrocław University
of Science and Technology

Outline

- 1 Introduction
- 2 Motzkin Neighborhood
- 3 Algorithm
- 4 Experiments
- 5 Conclusion

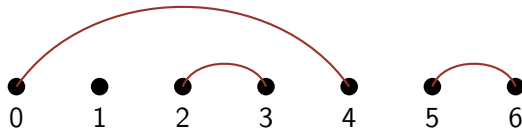
Introduction

- **Permutation Flow Shop Problem (PFSP):** n jobs processed in the same order on m machines; minimize $C_{\max}$.
- NP-hard for $m \geq 3$, motivating the use of metaheuristics.
- Solution quality depends not only on the algorithm, but also on the **neighborhood structure**.
- Classical neighborhoods (adjacent swap, Fibonacci) explore the search space locally and shallowly.
- **Composite neighborhoods** perform multiple swaps per iteration — potentially faster convergence at higher per-iteration cost.
- This work: a Motzkin-based composite neighborhood compared with Adjacent, Fibonacci, and Dynasearch under ILS and SA.

Motzkin Numbers and Neighborhood Structure

Motzkin numbers M_n count *non-crossing* arc configurations on n ordered points, allowing isolated points.

Example of an admissible composite move on 7 points — arcs $(0, 4)$, $(2, 3)$, $(5, 6)$:



Growth: $M_n \sim c \cdot 3^n / n^{3/2}$ — exponential, but dynamic programming avoids explicit enumeration.

Formal Definition

For permutation $\pi = (\pi_0, \dots, \pi_{n-1})$ and indices $0 \leq i < j < n$, the **end swap** exchanges $\pi_i \leftrightarrow \pi_j$, leaving intermediate elements unchanged.

A **composite move** $\mathcal{M} = \{(i_1, j_1), \dots, (i_k, j_k)\}$ is admissible iff:

- ❶ **Index disjointness:** $\{i_a, j_a\} \cap \{i_b, j_b\} = \emptyset$.
- ❷ **No crossings:** $i_a < i_b < j_a < j_b$ is forbidden.
- ❸ **Nesting allowed:** $i_a < i_b < j_b < j_a$ is admissible.

First Motzkin numbers:

n	2	3	4	5	6	8	10
M_n	2	4	9	21	51	323	2188

Each Motzkin number gives the size of the neighborhood for a permutation of length n .

Constructing the Best Composite Move

Single-iteration optimization via dynamic programming:

- 1 Compute Head and Tail completion-time matrices — $O(mn)$.
- 2 Evaluate all pairwise end-swap deltas $\Delta_{i,j}$ using Head+Tail — $O(mn^2)$.
- 3 DP picks the optimal set of non-crossing, non-overlapping pairs minimizing total Δ — $O(n^3)$.
- 4 Apply the chosen swaps and recompute $C_{\max}$.

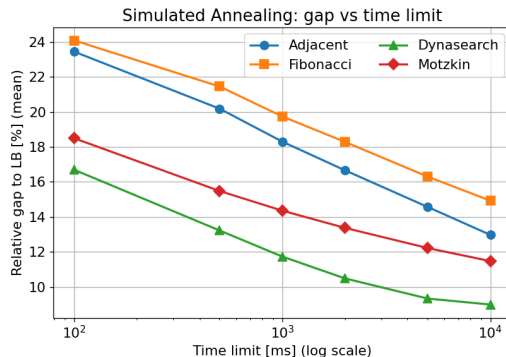
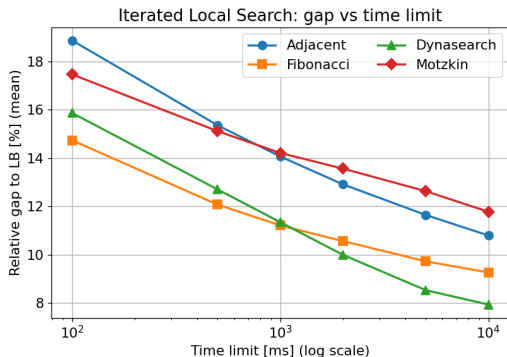
Trade-off:

- One Motzkin iteration explores up to M_n configurations in $O(n^3)$ time.
- One Adjacent iteration explores $n-1$ swaps in $O(mn)$ time.
- Fewer, richer moves vs. many, simple moves — balanced empirically.

Experimental Setup

- **Platform:** MacBook Pro M4 Pro, 24 GB RAM, macOS 26.0.1, Python 3.11.
- **Benchmarks:** Taillard instances with $n \in \{20, 50, 100, 200, 500\}$, $m \in \{5, 10, 20\}$ — 12 configurations, 10 instances each.
- **Time limits:** $t_{\max} \in \{100, 500, 1000, 2000, 5000, 10000\}$ ms.
- **Algorithms:**
 - **ILS** with tabu list, $\tau_{\text{tabu}} = 10$.
 - **SA:** $T_0 = 1000$, $T_{\min} = 1$, $\alpha = 0.95$, reheat $r = 1.5$, stagnation 500 ms.
- **Compared neighborhoods:** Adjacent, Fibonacci, Dynasearch, Motzkin.
- **Metric:** mean relative percentage deviation (RPD) to lower bound.

Results — Gap vs. Time Limit



- **Dynasearch** achieves the lowest gap across all budgets.
- **Motzkin** consistently outperforms Adjacent at larger $t_{\max}$.
- **Fibonacci** converges fastest but stagnates at the highest gap.

Statistical Analysis

Wilcoxon signed-rank test on mean gaps across 12 instance configurations:

- **Dynasearch** is significantly better than all other neighborhoods ($p < 0.01$, all $t_{\max}$).
- **Motzkin vs. Fibonacci**: significant for $t_{\max} \geq 5000$ ms ($p < 0.05$).
- **Motzkin vs. Adjacent**: not significant — two competing effects (Motzkin: few rich moves; Adjacent: many simple moves) tend to offset.

Observation: Fibonacci converges rapidly to a local optimum and then stagnates — adding an anti-stagnation mechanism is a promising direction.

Conclusion and Future Work

Summary:

- Motzkin gives a structured multi-swap neighborhood based on non-crossing arcs.
- DP enables $O(n^3)$ exploration of an exponentially large move set.
- Outperforms Adjacent at larger time budgets in both ILS and SA.
- Falls behind Dynasearch, which has a richer candidate set at lower cost.

Future work:

- Improve the computational efficiency of Motzkin and Fibonacci implementations to allow more iterations per budget.
- Quantum-assisted Motzkin evaluation (D-Wave QUBO, QAOA).
- Hybrid neighborhood strategies — cyclic / adaptive selection.